

Lecture 01:

Contents

1. [Variables](#)
 - Rules for choosing variables
 - Naming conventions
 - Creating variables
 - Deleting variables
 - Checking the types of variables
 - Reading variables
 - Printing variables
2. Some useful functions
 - [dir\(\)](#)
 - [help\(\)](#)
 - [type\(\)](#)
 - [id\(\)](#)
 - [print\(\)](#)
 - [input\(\)](#)
 - [eval](#)
3. [Summary](#)

Variables

In the early days of programming, programmers had to remember (memory addresses) where values are stored in memory in order to access them in code. Today programmers use names (of variables) to access values. These variables can be considered as containers in which values are stored. This is a huge improvement because for humans, a numeric address for a variable is much more difficult to work with than a descriptive identifier.

A variable is a named reference to a value stored in memory. It is like a container that holds data. The data can be (and is often) changed multiple times during program execution. The name of a variable should be descriptive, avoid single letter variables except when used as a iterator variable. Follow the coding standards of your organization.

Variables are locations in memory that can access via a name.

We will talk about scope (where they are accessible) of variables in week 5 when we talk about `functions` and mutability in week 6 when we cover `str`.

Unlike most other languages (JavaScript is a notable exception), python variables does not require explicit type declaration. Their type is inferred from the value assigned to them.

Rules for naming variables and or identifiers

An identifier is just a name to refer to something in your program, it could be the name of a variable, a function, a class or a module.

- Can contain letters, digits, and underscores **only**
- Start with a letter or underscore
- May not contain a python symbol (e.g. `,`, `:`, `.`, `->`, `(`, `)`, `[`, `]`, `{`, `}`, `@`, `...`)
- Cannot be a Python keyword (e.g. `if`, `else`, `elif`, `for`, `def`, `class`, `lambda`, `while`, `for`)

```
#valid names for variables
variable_1 = 10
_name = 'Narendra'
last_name = 'Pershad'
pi = 3.14
```

```
#invalid names for variables
# first name = 'Narendra'    #spaces are not allowed
# last-name = 'Pershad'      #hyphens are not allowed
# 1st_name = 'Narendra'      #cannot start with a number

# can you explain why the above variable names are invalid?
```

Styles (conventions) for naming variables in this course

We will cover general coding styles later in the course, but for now we will provide some guidelines on choosing names for variables.

Do:

- Use descriptive names
- Join multiple words with underscore `_`
- It is okay to use single letter variable for simple iterations

Avoid:

- Avoid double underscores at the start of an identifier `__` except when working with class members. The double underscores are used internally by python for its own identifiers.
- Avoid `l` (lowercase el) and `O` (uppercase oh), because on some OS's a user can be easily be confused with the digits 1 and 0.
- Non-Descriptive names

```
#prefer the following names
prof_name = 'Narendra Pershad'
source_file = 'variable_names.py'
```

```
#avoid the following names
# list = [1, 2, 3]
# str = 'Hello, World!'
# dict = {'key': 'value'}
```

```
# input = input('Enter your name: ')
# int = int(input('Enter a number: '))
# can you explain why the above variable names should be avoided?
```

Variables are created when you first assigned a value to it.

All variables must be initialized at declaration

Values are assign using the assignment (=) operator

e.g. `count = 5` # creates a variable called '**count**' and assigns the value 5 to it.

```
# Assigning values to variables
age = 25          # int
price = 19.99     # float
message = 'Hello!' # str
is_active = True  # bool

# Reassigning a variable/changed the value of a variable
price = 29.99     # Updated value
```

Variables are case-sensitive

`age`, `Age`, `AGE` are treated as different variables

```
age = 25
Age = 30
AGE = 35

print(age)    # 25
print(Age)    # 30
print(AGE)    # 35
25
30
35
```

Checking the type of a variables

Use the `type()` function to check the type of a variable

More data-types will be covered in details next week

```
# Checking the type of a variable
print(type(age))      # Output: <class 'int'>
print(type(price))    # Output: <class 'float'>
print(type(message))  # Output: <class 'str'>
print(type(is_active)) # Output: <class 'bool'>
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Reading variables

You can read the value of a variable by its name.

The python interpreter will go to the variable location, read the value and decode it properly.

Printing the value of a variable

Use the `print()` function to display the value of a variable

We will cover this function in more detail later

```
# Using variables in operations
discount = 0.1
cost = price * (1 - discount)
print(cost)                # Output: 26.991
print(price * (1 - discount)) # also produces the same output

print('You can suppress the automatic line feed', end='') #the next output
continues on this this line
print(', by utilizing the second argument of the print function.')
```



```
print(cost, age, message, is_active) # you may display multiple variable
with a single print call
26.991
26.991
You can suppress the automatic line feed, by utilizing the second argument of
the print function.
26.991 25 Hello! True
```

Delete or undeclare a variables

Use the `del` operator to remove a variable from the current environment

```
del cost # Deleting a variable
# Attempting to print a deleted variable will raise an error
# print(cost) # Uncommenting this line will raise a NameError
# variables are references to objects in memory
a = [10]
b = a
print(b)                # Output: 10
# Changing the value of 'a' does not affect 'b'
a = a.append(20)
print(b)                # Output: 10
[10]
[10, 20]
```

Some useful Functions

We will study functions later in this course, but for now you may think of it as something that can accomplish a specific task and a way to simplify coding tasks.

We will look at some function, that will be used frequently in this course.

These functions are built-in function in python i.e. they are available without doing any explicit imports.

dir()

The `dir()` function lists the attributes and methods of an object.

This is extremely useful for introspection. It helps you to explore what an object can do, especially in interactive environments like the Python REPL or Jupyter Notebook.

When called without argument it returns a list of names in the current local scope.

```
dir()
['AGE',
 'Age',
 'In',
 'Out',
 '_',
 '__',
 '___',
 '__builtin__',
 '__builtins__',
 '__doc__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 '__vsc_ipynb_file__',
 '_dh',
 '_i',
 '_i1',
 '_i2',
 '_i3',
 '_i4',
 '_i5',
 '_i6',
 '_i7',
 '_i8',
 '_i9',
 '_ih',
 '_ii',
 '_iii',
 '_name',
 '_oh',
 'a',
 'age',
 'b',
 'discount',
 'exit',
 'get_ipython',
 'is_active',
 'last_name',
 'message',
 'open',
 'pi',
 'price',
 'prof_name',
```

```

'quit',
'source_file',
'variable_1']
# returns a list of names associated with the argument
dir(str)      # Output: List of attributes and methods of the str class
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getitem__',
 '__getnewargs__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mod__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__rmod__',
 '__rmul__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
'capitalize',
'casefold',
'center',
'count',
'encode',
'endswith',
'expandtabs',
'find',
'format',
'format_map',
'index',
'isalnum',
'isalpha',
'isascii',

```

```
'isdecimal',
'isdigit',
'isidentifier',
'islower',
'isnumeric',
'isprintable',
'isspace',
'istitle',
'isupper',
'join',
'ljust',
'lower',
'lstrip',
'maketrans',
'partition',
'removeprefix',
'removesuffix',
'replace',
'rfind',
'rindex',
'rjust',
'rpartition',
'rsplit',
'rstrip',
'split',
'splitlines',
'startswith',
'strip',
'swapcase',
'title',
'translate',
'upper',
'zfill']
```

help()

The `help()` function provides interactive access to Python's documentation (docstrings). It is extremely useful for getting quick information about modules, classes, functions, keywords, and other Python objects.

When called without arguments it enters the interactive help module

```
help()
```

```
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.
```

```
Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules. To get a list of available
modules, keywords, symbols, or topics, enter "modules", "keywords",
"symbols", or "topics".
```

Each module also comes with a one-line summary of what it does; to list

the modules whose name or summary contain a given string such as "spam", enter "modules spam".

To quit this help utility and return to the interpreter, enter "q", "quit" or "exit".

displays the documentation for the str class

help(str)

Help on class str in module builtins:

```
class str(object)
|   str(object='') -> str
|   str(bytes_or_buffer[, encoding[, errors]]) -> str
|
|   Create a new string object from the given object. If encoding or
|   errors is specified, then the object must expose a data buffer
|   that will be decoded using the given encoding and error handler.
|   Otherwise, returns the result of object.__str__() (if defined)
|   or repr(object).
|   encoding defaults to 'utf-8'.
|   errors defaults to 'strict'.
|
|   Methods defined here:
|
|   __add__(self, value, /)
|       Return self+value.
|
|   __contains__(self, key, /)
|       Return bool(key in self).
|
|   __eq__(self, value, /)
|       Return self==value.
|
|   __format__(self, format_spec, /)
|       Return a formatted version of the string as described by format_spec.
|
|   __ge__(self, value, /)
|       Return self>=value.
|
|   __getitem__(self, key, /)
|       Return self[key].
|
|   __getnewargs__(self, /)
|
|   __gt__(self, value, /)
|       Return self>value.
|
|   __hash__(self, /)
|       Return hash(self).
|
|   __iter__(self, /)
|       Implement iter(self).
```



```

|  __le__(self, value, /)
|      Return self<=value.
|
|  __len__(self, /)
|      Return len(self).
|
|  __lt__(self, value, /)
|      Return self<value.
|
|  __mod__(self, value, /)
|      Return self%value.
|
|  __mul__(self, value, /)
|      Return self*value.
|
|  __ne__(self, value, /)
|      Return self!=value.
|
|  __repr__(self, /)
|      Return repr(self).
|
|  __rmod__(self, value, /)
|      Return value%self.
|
|  __rmul__(self, value, /)
|      Return value*self.
|
|  __sizeof__(self, /)
|      Return the size of the string in memory, in bytes.
|
|  __str__(self, /)
|      Return str(self).
|
|  capitalize(self, /)
|      Return a capitalized version of the string.
|
|      More specifically, make the first character have upper case and the r
est lower
|      case.
|
|  casefold(self, /)
|      Return a version of the string suitable for caseless comparisons.
|
|  center(self, width, fillchar=' ', /)
|      Return a centered string of length width.
|
|      Padding is done using the specified fill character (default is a spac
e).
|
|  count(self, sub[, start[, end]], /)
|      Return the number of non-overlapping occurrences of substring sub in
string S[start:end].
|

```

```

|         Optional arguments start and end are interpreted as in slice notation
.
|
| encode(self, /, encoding='utf-8', errors='strict')
|     Encode the string using the codec registered for encoding.
|
|     encoding
|         The encoding in which to encode the string.
|     errors
|         The error handling scheme to use for encoding errors.
|         The default is 'strict' meaning that encoding errors raise a
|         UnicodeEncodeError. Other possible values are 'ignore', 'replace'
and
|         'xmlcharrefreplace' as well as any other name registered with
|         codecs.register_error that can handle UnicodeEncodeErrors.
|
|     endswith(self, suffix[, start[, end]], /)
|         Return True if the string ends with the specified suffix, False other
wise.
|
|         suffix
|             A string or a tuple of strings to try.
|         start
|             Optional start position. Default: start of the string.
|         end
|             Optional stop position. Default: end of the string.
|
|     expandtabs(self, /, tabsize=8)
|         Return a copy where all tab characters are expanded using spaces.
|
|         If tabsize is not given, a tab size of 8 characters is assumed.
|
|     find(self, sub[, start[, end]], /)
|         Return the lowest index in S where substring sub is found, such that
sub is contained within S[start:end].
|
|         Optional arguments start and end are interpreted as in slice notation
.
|         Return -1 on failure.
|
|     format(self, /, *args, **kwargs)
|         Return a formatted version of the string, using substitutions from ar
gs and kwargs.
|         The substitutions are identified by braces ('{' and '}').
|
|     format_map(self, mapping, /)
|         Return a formatted version of the string, using substitutions from ma
pping.
|         The substitutions are identified by braces ('{' and '}').
|
|     index(self, sub[, start[, end]], /)
|         Return the lowest index in S where substring sub is found, such that
sub is contained within S[start:end].

```

```

|
|     Optional arguments start and end are interpreted as in slice notation
.
|     Raises ValueError when the substring is not found.
|
|     isalnum(self, /)
|         Return True if the string is an alpha-numeric string, False otherwise
.
|
|     A string is alpha-numeric if all characters in the string are alpha-n
numeric and
|         there is at least one character in the string.
|
|     isalpha(self, /)
|         Return True if the string is an alphabetic string, False otherwise.
|
|     A string is alphabetic if all characters in the string are alphabetic
and there
|         is at least one character in the string.
|
|     isascii(self, /)
|         Return True if all characters in the string are ASCII, False otherwis
e.
|
|         ASCII characters have code points in the range U+0000-U+007F.
|         Empty string is ASCII too.
|
|     isdecimal(self, /)
|         Return True if the string is a decimal string, False otherwise.
|
|     A string is a decimal string if all characters in the string are deci
mal and
|         there is at least one character in the string.
|
|     isdigit(self, /)
|         Return True if the string is a digit string, False otherwise.
|
|     A string is a digit string if all characters in the string are digits
and there
|         is at least one character in the string.
|
|     isidentifier(self, /)
|         Return True if the string is a valid Python identifier, False otherwi
se.
|
|         Call keyword.iskeyword(s) to test whether string s is a reserved iden
tifier,
|         such as "def" or "class".
|
|     islower(self, /)
|         Return True if the string is a lowercase string, False otherwise.
|

```

```

|         A string is lowercase if all cased characters in the string are lower
case and
|         there is at least one cased character in the string.
|
|     isnumeric(self, /)
|         Return True if the string is a numeric string, False otherwise.
|
|         A string is numeric if all characters in the string are numeric and t
here is at
|         least one character in the string.
|
|     isprintable(self, /)
|         Return True if all characters in the string are printable, False othe
rwise.
|
|         A character is printable if repr() may use it in its output.
|
|     isspace(self, /)
|         Return True if the string is a whitespace string, False otherwise.
|
|         A string is whitespace if all characters in the string are whitespace
and there
|         is at least one character in the string.
|
|     istitle(self, /)
|         Return True if the string is a title-cased string, False otherwise.
|
|         In a title-cased string, upper- and title-case characters may only
follow uncased characters and lowercase characters only cased ones.
|
|     isupper(self, /)
|         Return True if the string is an uppercase string, False otherwise.
|
|         A string is uppercase if all cased characters in the string are upper
case and
|         there is at least one cased character in the string.
|
|     join(self, iterable, /)
|         Concatenate any number of strings.
|
|         The string whose method is called is inserted in between each given s
tring.
|         The result is returned as a new string.
|
|         Example: '.'.join(['ab', 'pq', 'rs']) -> 'ab.pq.rs'
|
|     ljust(self, width, fillchar=' ', /)
|         Return a left-justified string of length width.
|
|         Padding is done using the specified fill character (default is a spac
e).
|
|     lower(self, /)

```

```

|         Return a copy of the string converted to lowercase.
|
|     lstrip(self, chars=None, /)
|         Return a copy of the string with leading whitespace removed.
|
|         If chars is given and not None, remove characters in chars instead.
|
|     partition(self, sep, /)
|         Partition the string into three parts using the given separator.
|
|         This will search for the separator in the string.  If the separator i
s found,
|         returns a 3-tuple containing the part before the separator, the separ
ator
|         itself, and the part after it.
|
|         If the separator is not found, returns a 3-tuple containing the origi
nal string
|         and two empty strings.
|
|     removeprefix(self, prefix, /)
|         Return a str with the given prefix string removed if present.
|
|         If the string starts with the prefix string, return string[len(prefix
):].
|         Otherwise, return a copy of the original string.
|
|     removesuffix(self, suffix, /)
|         Return a str with the given suffix string removed if present.
|
|         If the string ends with the suffix string and that suffix is not empt
y,
|         return string[:-len(suffix)]. Otherwise, return a copy of the origina
l
|         string.
|
|     replace(self, old, new, /, count=-1)
|         Return a copy with all occurrences of substring old replaced by new.
|
|         count
|             Maximum number of occurrences to replace.
|             -1 (the default value) means replace all occurrences.
|
|         If the optional argument count is given, only the first count occurre
nces are
|         replaced.
|
|     rfind(self, sub[, start[, end]], /)
|         Return the highest index in S where substring sub is found, such that
sub is contained within S[start:end].
|
|         Optional arguments start and end are interpreted as in slice notation
.

```

```

|         Return -1 on failure.
|
|     rindex(self, sub[, start[, end]], /)
|         Return the highest index in S where substring sub is found, such that
sub is contained within S[start:end].
|
|         Optional arguments start and end are interpreted as in slice notation
.
|         Raises ValueError when the substring is not found.
|
|     rjust(self, width, fillchar=' ', /)
|         Return a right-justified string of length width.
|
|         Padding is done using the specified fill character (default is a spac
e).
|
|     rpartition(self, sep, /)
|         Partition the string into three parts using the given separator.
|
|         This will search for the separator in the string, starting at the end
. If
|         the separator is found, returns a 3-tuple containing the part before
the
|         separator, the separator itself, and the part after it.
|
|         If the separator is not found, returns a 3-tuple containing two empty
strings
|         and the original string.
|
|     rsplit(self, /, sep=None, maxsplit=-1)
|         Return a list of the substrings in the string, using sep as the separ
ator string.
|
|         sep
|         The separator used to split the string.
|
|         When set to None (the default value), will split on any whitespac
e
|         character (including \n \r \t \f and spaces) and will discard
|         empty strings from the result.
|         maxsplit
|         Maximum number of splits.
|         -1 (the default value) means no limit.
|
|         Splitting starts at the end of the string and works to the front.
|
|    rstrip(self, chars=None, /)
|         Return a copy of the string with trailing whitespace removed.
|
|         If chars is given and not None, remove characters in chars instead.
|
|     split(self, /, sep=None, maxsplit=-1)

```

```

|         Return a list of the substrings in the string, using sep as the separ
ator string.
|
|         sep
|         The separator used to split the string.
|
|         When set to None (the default value), will split on any whitespac
e
|         character (including \n \r \t \f and spaces) and will discard
|         empty strings from the result.
|         maxsplit
|         Maximum number of splits.
|         -1 (the default value) means no limit.
|
|         Splitting starts at the front of the string and works to the end.
|
|         Note, str.split() is mainly useful for data that has been intentional
ly
|         delimited. With natural text that includes punctuation, consider usi
ng
|         the regular expression module.
|
|         splitlines(self, /, keepends=False)
|         Return a list of the lines in the string, breaking at line boundaries
.
|
|         Line breaks are not included in the resulting list unless keepends is
given and
|         true.
|
|         startswith(self, prefix[, start[, end]], /)
|         Return True if the string starts with the specified prefix, False oth
erwise.
|
|         prefix
|         A string or a tuple of strings to try.
|         start
|         Optional start position. Default: start of the string.
|         end
|         Optional stop position. Default: end of the string.
|
|         strip(self, chars=None, /)
|         Return a copy of the string with leading and trailing whitespace remo
ved.
|
|         If chars is given and not None, remove characters in chars instead.
|
|         swapcase(self, /)
|         Convert uppercase characters to lowercase and lowercase characters to
uppercase.
|
|         title(self, /)
|         Return a version of the string where each word is titlecased.

```

```

|
|     More specifically, words start with uppercased characters and all rem
aining
|     cased characters have lower case.
|
|     translate(self, table, /)
|         Replace each character in the string using the given translation tabl
e.
|
|         table
|             Translation table, which must be a mapping of Unicode ordinals to
|             Unicode ordinals, strings, or None.
|
|         The table must implement lookup/indexing via __getitem__, for instanc
e a
|         dictionary or list.  If this operation raises LookupError, the charac
ter is
|         left untouched.  Characters mapped to None are deleted.
|
|     upper(self, /)
|         Return a copy of the string converted to uppercase.
|
|     zfill(self, width, /)
|         Pad a numeric string with zeros on the left, to fill a field of the g
iven width.
|
|         The string is never truncated.
|
|     -----
|     Static methods defined here:
|
|     __new__(*args, **kwargs)
|         Create and return a new object.  See help(type) for accurate signatur
e.
|
|     maketrans(x, y=<unrepresentable>, z=<unrepresentable>, /)
|         Return a translation table usable for str.translate().
|
|         If there is only one argument, it must be a dictionary mapping Unicod
e
|         ordinals (integers) or characters to Unicode ordinals, strings or Non
e.
|         Character keys will be then converted to ordinals.
|         If there are two arguments, they must be strings of equal length, and
|         in the resulting dictionary, each character in x will be mapped to th
e
|         character at the same position in y.  If there is a third argument, it
|         must be a string, whose characters will be mapped to None in the resu
lt.
|
|
|     # displays the help on the upper method of the str class
help(str.upper)
Help on method_descriptor:

```



```
upper(self, /) unbound builtins.str method
    Return a copy of the string converted to uppercase.
```

A very useful way to browse all of the installed python libraries, is to use the following command:

```
python -m pydoc -b
```

This will create a virtual local web site that will allow you to browse the various attribute and methods from your web browser.

If you want to just browse a particular topic, then the following command will suffice

```
python -m pydoc str|int|dict
```

```
python -m pydoc -n <hostname> will start the server listening at the given hostname
```

```
python -m pydoc -p 1234 will start the server listening at the given port
```

type()

`type()` returns the datatype (class) of the argument

This function can also be used to dynamically create a class. (Advanced usage)

```
# Checking the type of a variable
print(type(age))      # Output: <class 'int'>
print(type(price))    # Output: <class 'float'>
print(type(message))  # Output: <class 'str'>
print(type(is_active)) # Output: <class 'bool'>
print(type(None))     # Output: <class 'NoneType'>

print(type([1, 2, 3])) # Output: <class 'list'>
print(type((1, 2, 3))) # Output: <class 'tuple'>
print(type({'1': 'one'})) # Output: <class 'dict'>
print(type({1, 2, 3})) # Output: <class 'set'>

print(type(type(1)))
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
<class 'type'>
```

The `issubclass()` and `isinstance()` are also useful to check the type of an object.

id()

In our implementation of python, `id()` corresponds to the memory address of the object

No two objects will have the same `id()`, at the same time (unless they are the same object)

If two variables reference the same object, their `id()` will be the same

```
a = 10
b = a

print(id(a))

print(id(b))  # Should be the same as id(a) since b references the same
              # object
140704902444232
140704902444232
# the is operator checks if two variables point to the same object
print(a is b)  # Output: True
True
c = 11
c = 10
print(a is c)  # Output: True, because c was reassigned to the same value as
a
True
```

input()

The `input()` function is used to take user input while the program is running.

- It always returns a string even if the user types a number!
- If you want an int or float, then you will have to do the necessary conversion.
- It takes an optional message that is displayed before the input is taken.
- When called, it pauses program execution and waits for the user to type something.

```
name = input('Enter your name: ')
print(name)  # Output: whatever the user inputs

age = int(input('Enter your age: '))
print(age)  # Output: whatever the user inputs, converted to an integer
# Note: If the user inputs a non-integer value, this will raise a ValueError
```

eval()

`eval()` is a powerful but dangerous function - use with utmost caution.

It evaluates strings as python code and returns the result.

It should be avoided at all cost in production code.

However, we will use this function quite frequently in our notebooks because it demonstrates some features of code exquisitely.

```
user_input = '2 + 3'
data = eval(user_input)
```

```

print(f'Orig type: {type(user_input)}, resulting type: {type(data)}')
print(data)
Orig type: <class 'str'>, resulting type: <class 'int'>
5
user_input = '{"name": "Alice", "age": 30}'
data = eval(user_input) # Converts to dict (but prefer `json.loads()`)
print(f'Orig type: {type(user_input)}, resulting type: {type(data)}')
print(data) # Output: {'name': 'Alice', 'age': 30}
Orig type: <class 'str'>, resulting type: <class 'dict'>
{'name': 'Alice', 'age': 30}

```

Summary

- **Variables:** Variables are named references to values stored in memory, acting like containers that hold data which can change during program execution.
 - **Rules:** Variable names (identifiers) must start with a letter or underscore, can include letters, digits, and underscores, but cannot contain spaces, operators, symbols, or be Python keywords.
 - **Conventions:** Use descriptive names, separate words with underscores, avoid confusing characters like l and O, and reserve double underscores for special class-related use.
 - **Case Sensitivity:** Python treats variable names as case-sensitive, so age, Age, and AGE are considered different variables.
 - **Initialization:** Variables are created and initialized when a value is first assigned using the = operator.
- **Dynamic Typing:** Python does not require explicit type declarations; the type is inferred from the assigned value.
- **Reading and Printing:** Use the variable name to read its value and print() to display it. The interpreter fetches and decodes the value from memory.

Useful Built-in Functions:

- `input()` gets user input (always returns a string).
- `print()` sends text to the screen.
- `dir()` lists attributes/methods of objects.
- `help()` accesses documentation.
- `eval()` executes a string as Python code (use cautiously).
- `type()` to check a variable's data type.
- `id()` to get the memory address of an object.